A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js

Mateusz Miotk
Faculty of Mathematics, Physics and Informatics,
University of Gdańsk
mateusz.miotk@ug.edu.pl

July 22, 2026

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks whose numbers are rarely shown to be comparable — fragmented, PostgreSQL-only, and seldom reporting the tail (p99). **Objective:** We contribute a reusable *comparability protocol* for access-layer benchmarking — a semantic-equivalence correctness gate, a documentation-selected estimand paired with a same-SQL standardized contrast, and separated operating points — realized in an open harness and demonstrated through a dual-engine PostgreSQL/MySQL case study. The protocol, not any ranking, is the contribution. **Methods:** An identical Express application is served through nine documentation-selected access layers (native drivers `pg` and `mysql2`, the query builder Knex, and the ORMs Drizzle, Prisma, Sequelize, TypeORM, Objection and MikroORM) and two tuned native baselines, over five access patterns, on the latest stable library versions current at the July 2026 freeze. We report throughput with the *high-load response-time p99 under equal closed-loop demand* (a fixed 50-connection point) from 25 repeated runs per cell, and on the deep fetch distinguish it from the tail at *equal utilization* (an open-loop sweep, where the gap converges — the near-saturation p99 chiefly a capacity-and-queueing outcome, not a sub-saturation latency difference) and an exploratory equal compute budget; an automated cross-check verifies

byte-identical responses before measurement, and inserts run under the engines' default durability. **Results:** The implementation-and-strategy difference is largest on the tested deep/nested fetch: spreads 7.15× (PostgreSQL) and 4.96× (MySQL) between native and the slowest ORM, against only 1.68× and 2.01× among the raw paths running common SQL, a standardized contrast that isolates no single mechanism. The read ordering holds across engines, but the aggregation and insert orderings reverse: Prisma drops to last on the MySQL insert, and the per-layer engine advantage scatters 1.6–3.2×. No layer exceeds ≈ 110% of one core; the native driver leads all ten engine-pattern combinations. **Conclusion:** The case study shows why the protocol matters: access-layer rankings are configuration-specific and version-sensitive, so the durable contribution is the protocol and its harness, not any single ranking.

Keywords: benchmarking methodology; database access layer; object-relational mapping; Node.js; response-time tail (p99); empirical software engineering

1 Introduction

Express.js is among the most widely used web frameworks for Node.js, and almost every non-trivial service built on it reaches a relational database through some *access layer*. Access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a nested fetch issues one query per parent row instead of one for the result graph [9] — a pervasive anti-pattern (Section 2). Native drivers expose the wire protocol directly (`pg`, `mysql2`); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects, following patterns such as Data Mapper and Active Record [21]. Node.js back-end performance under load is a recognized empirical topic [6], [7], [36], but that literature varies the runtime and framework, not the access layer.

The most visible comparisons are *vendor benchmarks*: plentiful but narrow, published by the maintainers of the compared products; in the sources searched, none reports the p99 tail that governs user-perceived latency under load [19], [22]. The peer-reviewed literature is sparser still: at most three ORMs, confined to PostgreSQL, and, where it compares the two engines directly, holding the access layer fixed rather than varying it [2], [15], [50], [62], [63] (Section 2 surveys each).

This leaves a gap along the four coverage axes of Table 1 — access-layer taxonomy, engine, joint throughput/tail metrics, and vendor involvement: no access-layer comparison includes MySQL; no PostgreSQL/MySQL benchmark varies the access layer; the only broad-taxonomy comparisons are vendor-authored, academic work covering at most three libraries [2], [63]; and the sole vendor suite reporting a tail stops at one P95 at one load point [20]. None measures client-observed performance through an HTTP framework; academic studies instrument query execution inside the process [29], [63].

These are gaps in *coverage*. A more basic gap is methodological: the reported numbers are not established to be *comparable* — a layer returning fewer fields or erroring faster than it works can look faster, a library's chosen query strategy is conflated with its own cost, and a single, arbitrary load point conflates capacity, tail latency under demand, and per-request latency. Establishing that comparability is the problem this paper takes up.

This paper addresses these gaps with a *comparability protocol* and a non-vendor-authored case study that applies it (Section 3 defines the term): an *identical* Express application served through all nine access layers (two native drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five representative access patterns — point read, keyset range scan, deep/nested fetch, per-author aggregation, and insert — rather than a production workload. Seven layers run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines mark what hand-tuned driver use reaches. Every (layer, engine, pattern) reports throughput (req/s) and the *high-load response-time p99 under equal closed-loop demand* at the fixed 50-connection operating point, over 25 repeated runs. At this near-saturation point the p99 largely tracks capacity and queueing, so a lower-capacity layer looks worse on the tail even when its sub-saturation latency is not; we therefore also report the tail at *matched utilization*. The study separates two estimands: the *documentation-selected implementation-and-strategy* effect of each layer (RQ1–RQ3), and a controlled *same-SQL* standardized contrast reporting the residual under identical SQL — changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness — adapters, seed data, an autocannon-driven [16] load runner, and an automated *byte-identical* cross-check on the four non-mutating patterns — is released as an open replication package.[1]

The case study answers three research questions, each scoped to the bench-

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.12.11 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21497964`.

marked implementations under the specified operating point:

RQ1 *(performance difference):* How large is the throughput and response-time-p99 *difference* of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities a fixed operating point conflates: *capacity* (saturating throughput), *high-load latency* (p99 under equal demand), and *latency at matched utilization* (equal fractions of each layer's own capacity).

RQ2 *(engine dependence):* Does the observed layer ordering transfer across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(pattern spread):* Which of the five tested patterns shows the largest native-relative spread in this configuration?

This paper's primary contribution is a **comparability protocol for access-layer benchmarking**, specified normatively in Section 3.1 (inputs, stages, cell-admission criteria, interpretation, and limits): a set of controls that turn the vendor-benchmark genre into a controlled experiment by establishing three preconditions it otherwise leaves unmet.

- **Semantic equivalence.** Every layer must return *byte-identical* results on the conformance suite (fixed probes plus a seeded property-based sweep), and every write must produce the intended database state — field values, generated identifiers, exact row-count changes, and transactional rollback, not merely a 2xx — before its timing is admitted; a layer that returns less, writes wrong, or errors faster than it works thus cannot appear faster (Section 3).

- **Strategy standardization.** The treatment is fixed by a predeclared, reproducible selection rule (*documentation-selected* here; the protocol privileges none), and a *same-SQL standardized contrast* reports how much difference remains once SQL is held fixed — a diagnostic contrast, not a causal attribution.

- **Operating-point separation.** Capacity, high-load tail under equal demand, and tail at matched utilization are reported as *distinct* quantities, not conflated at one load point.

These preconditions are not new: correctness, workload equivalence, warm-up, coordinated-omission correction, and reproducibility are long established in performance evaluation (Section 2) [22], [24], [47]; the contribution is their *domain-specific synthesis and operationalization* for the access-layer genre —

which assumes comparability rather than establishing it — not their invention. The **artifact** — an open, reusable harness — *operationalizes* the protocol; the **empirical case study** — eleven implementations across PostgreSQL and MySQL (Section 4) — *demonstrates* it and shows each precondition is load-bearing: a broken MikroORM cell briefly *led* the MySQL write ranking until the equivalence gate excluded it; the sevenfold documentation-selected deep-fetch spread was only 1.7× among raw paths running common SQL; and a large high-load tail gap dissolved at matched utilization. The resulting rankings are configuration-specific and version-sensitive; the case study also yields a checklist of genre-specific benchmarking pitfalls (Section 5).

2 Related Work

We survey prior work along four *coverage* axes — taxonomy, engine, joint throughput/tail reporting, and vendor involvement — our case study spans (Table 1); it leaves the comparability of its numbers unestablished, the gap our protocol addresses. A structured scoping search (not a systematic review) of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (2006–July 2026) retained empirical comparisons of relational access layers or engines, with search details archived in the replication package (`notes/related-work-search.md`); being scoping rather than exhaustive, we scope every gap claim to the sources searched.

2.1 Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and an application's object graphs [30], spanning native drivers, query builders, and ORMs following Data Mapper / Active Record patterns [21] (Section 1). Torres et al. [55] survey two decades of ORM patterns; their trade-off — productivity against a hard-to-predict runtime cost — is what we quantify, holding the store choice (relational vs. non-relational [49]) fixed while varying only the access layer.

2.2 ORM performance and data-access anti-patterns

A parallel line studies ORM performance in reverse, repairing the inefficient SQL frameworks emit. The costliest is the N+1 query problem (Section 1); it and related redundant-query anti-patterns pervade real-world web applications [52], [59], [60], their impact quantified in ORM-backed systems [9],

[10] and often *removable* by query synthesis, batching, or caching [8], [11], [12] (with a JavaScript `async`/`await` analogue [56]). Closest to our design, Lorenz et al. [43] show the best mapping strategy depends on database technology, van Zyl et al. [64] that it is tuning-sensitive, and a recent study adds an energy dimension [5]. Overhead is thus configurable, not fixed — as our per-pattern results confirm — but this line never compares drivers, query builders, and ORMs head-to-head under load, our aim.

2.3 Peer-reviewed access-layer comparisons

Peer-reviewed head-to-head access-layer measurement is scarce. Colley et al. [15] give the earliest cross-language account — eight ORMs across Java, C#, Python, and PHP on a common schema — but *deliberately exclude JavaScript*, leaving it without an academic baseline for nearly a decade. Three recent studies partially address this: one in *Procedia Computer Science* compares Prisma against optimized raw SQL over eight query patterns on PostgreSQL [62] (differing versions, hardware, and workload, so we neither build on nor dispute its figures); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [2]; the most recent times internal query stages of the same three ORMs rather than client-observed requests [63], independently observing the concurrency-dependent Prisma ranking flip we also report. A further Express study optimizes one stack rather than comparing access layers [29].

2.4 Database-engine, SQL/NoSQL, and Node.js performance

A second, disjoint line benchmarks the layers below and above the access layer without varying it. Salunke and Ouda [50] compare PostgreSQL and MySQL under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, so cannot say how an access layer changes it; SQL/NoSQL comparisons [42] likewise measure the store, not the layer. On the Node.js side, work establishes Node's web viability [7], tracks back-end drift under sustained load [36], and compares frameworks broadly [6], but none treats the access layer as an independent variable across the taxonomy studied here.

2.5 Standard benchmarks and benchmarking methodology

Standard benchmarks define how systems are measured: YCSB [18] for cloud-serving stores and TPC-C/TPC-E for OLTP. These target the *engine*, not the driver, query builder, or ORM this study varies, but set the bar for controlled, repeatable load. A methodological literature explains why single-metric reporting fails: Fruth et al. [22] and Raasveldt et al. [47] catalog benchmarking pitfalls (coordinated omission, warm-up); Dean and Barroso [19] show the tail, not the mean, drives user-perceived latency under fan-out; and Li et al. [41] trace its sources. Such measurement is often ad hoc [31], [40] and hard to reproduce [14], motivating a harness built for relevance, repeatability, and fairness [28], [53]. We take a choice absent above: report throughput *and* p99 for the same run, not a single after-the-fact metric.

2.6 Practitioner and vendor benchmarks

A larger body comes from practitioners and vendors, each authored by a party with a stake in a compared product. Drizzle publishes a Northwind micro-suite (nine targets) and a k6 load test reporting req/s, average latency, P95, and CPU [20]. Prisma's site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any percentile [46]. IMDBench adds Sequelize and node-postgres, reporting throughput and latency on three CRUD operations [23]. Together these cover more taxonomy than the peer-reviewed literature combined, yet share the defects of Section 1: vendor authorship, PostgreSQL-only coverage, and a single metric family.

2.7 Positioning

In the sources searched, access-layer benchmarks report numbers whose comparability is assumed, not established, and we found no study combining broad taxonomy, both engines, joint throughput/tail reporting, and non-vendor authorship — each covers at most two or three (Table 1), the strongest claim the scoping search supports, not one of priority. But coverage is not the contribution: the protocol operationalizes established experimental principles for this genre, not a new one. Kounev et al.'s systems-benchmarking criteria [34], Hasselbring's software-engineering standard [27], and Raasveldt and Mühleisen's fair-benchmarking guidance [47] supply its principles, but abstractly: none prescribes a documentation-selected, predeclared rule for *what code represents a library* or a same-SQL contrast holding the query

strategy fixed, and their correctness norms ("verify results", construct validity) stop short of a per-cell admission gate requiring byte-identical responses and verified write-state *before* timing. Correctness-before-timing has precedents (SPEC validation, TPC ACID, SQLancer's equivalence oracles [48]), but each validates one program against its own reference, not equivalence *across competing implementations of one task*. Together these three constructs — predeclared treatment-selection, cross-implementation admission-gated equivalence, and a same-SQL diagnostic — yield one domain-specific property general frameworks leave to the practitioner: an admission-and-attribution discipline making *competing implementations of one task* mutually comparable. That domain specialization, realized in the open harness (Section 1), is the contribution, not the invention of the underlying principles; a post-freeze pre-submission search surfaced no new prior art.

Table 1: Prior work on relational-database access-layer performance, positioned along four *coverage* axes our case study spans; the paper's contribution is the comparability protocol of Section 1, not this coverage. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer. *Vendor involvement* marks maintainer-authored studies — a conflict-of-interest property of the source, not a methodological guarantee that a non-vendor-authored benchmark is unbiased.

Study	Layers covered	Engines	Metrics	Vendor involvement
Colley et al. [15]	8 ORMs, 4 languages (no JS)	not stated	query latency	None
Procedia CS [62]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	None
JCCT [2]	3 ORMs	PostgreSQL	latency, memory, CPU	None
JCSI [63]	3 ORMs	PostgreSQL	per-stage latency	None
Salunke and Ouda [50]	none (engine only)	PostgreSQL + MySQL	throughput, latency	None
Drizzle [20]	9 (broad)	PostgreSQL	throughput, latency, P95	Vendor
Prisma [46]	3 ORMs	PostgreSQL	median latency only	Vendor
IMDBench [23]	4 ORMs + driver	PostgreSQL	throughput, latency	Vendor
This study	**9 documentation-selected (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**None**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

3 Study Design

In Goal–Question–Metric terms the goal is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying its documentation-selected strategy's performance *with respect to* throughput and response-time p99 *from the point of view of* a developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [4]. It is a controlled benchmark *case study* of the tested implementations under one synthetic application and working-set regime, not statistical inference about ORMs

as a population. This section specifies the study's *comparability protocol* (Section 1) — semantic-equivalence cross-check, documentation-selected estimand, separated operating conditions — and the dual-engine case study applying it, following guidelines for controlled software-engineering experiments [33], [58]. The **primary comparison** (RQ1–RQ3) reads each layer's *documentation-selected implementation and query/loading strategy* (the API its documentation presents first, not the most common or performant one); a **same-SQL (raw-path) standardized contrast** then reports the residual once every layer runs identical SQL through its raw facility. The **mechanisms** differing within a layer — eager loading, wire protocol, round-trips, statement preparation, hydration — change together and are *not* separately or causally identified; the same-SQL contrast is a standardized diagnostic, not a decomposition and not a bound on the intrinsic library effect (Supplement Table S42 enumerates the components it moves).

A fixed concurrency imposes equal *demand* on layers of unequal throughput and so drives them to unequal *utilization*, so RQ1 is characterized under three operating conditions rather than one privileged point — the protocol's *operating-point separation*. *Equal external demand* is the fixed 50-connection *high-load* point (the full five-pattern, two-engine matrix); a per-pattern concurrency sweep (ladder 1–200, both engines; Supplement Table S35) puts each pattern's throughput knee at or below 50 connections, so this point places every pattern at high utilization of its own capacity (median 97–100%) — a capacity-and-queueing, sub-saturation-latency regime, not an intrinsic tail — while 50 connections against a ten-connection pool keep about forty requests queued (Controls, below). Capacity is thus *measured* for all five patterns (deep-fetch curve, Supplement Figure S2). The two utilization-dependent conditions need a per-layer *saturating-throughput* target and are shown on the deep fetch: *equal utilization* is the coordinated-omission-corrected open-loop sweep offering each layer a fixed fraction (50/70/85/95%) of its *own* saturating throughput, and an *equal compute budget* is the exploratory equal-CPU check — different questions that need not agree, so all three are reported.

3.1 The comparability protocol

We state the protocol independently of the case study, then instantiate it. It turns *treatments* compared on a fixed *workload* into a controlled experiment; Figure 1 states its inputs, mandatory and recommended stages, cell-admission rule, and output interpretation.

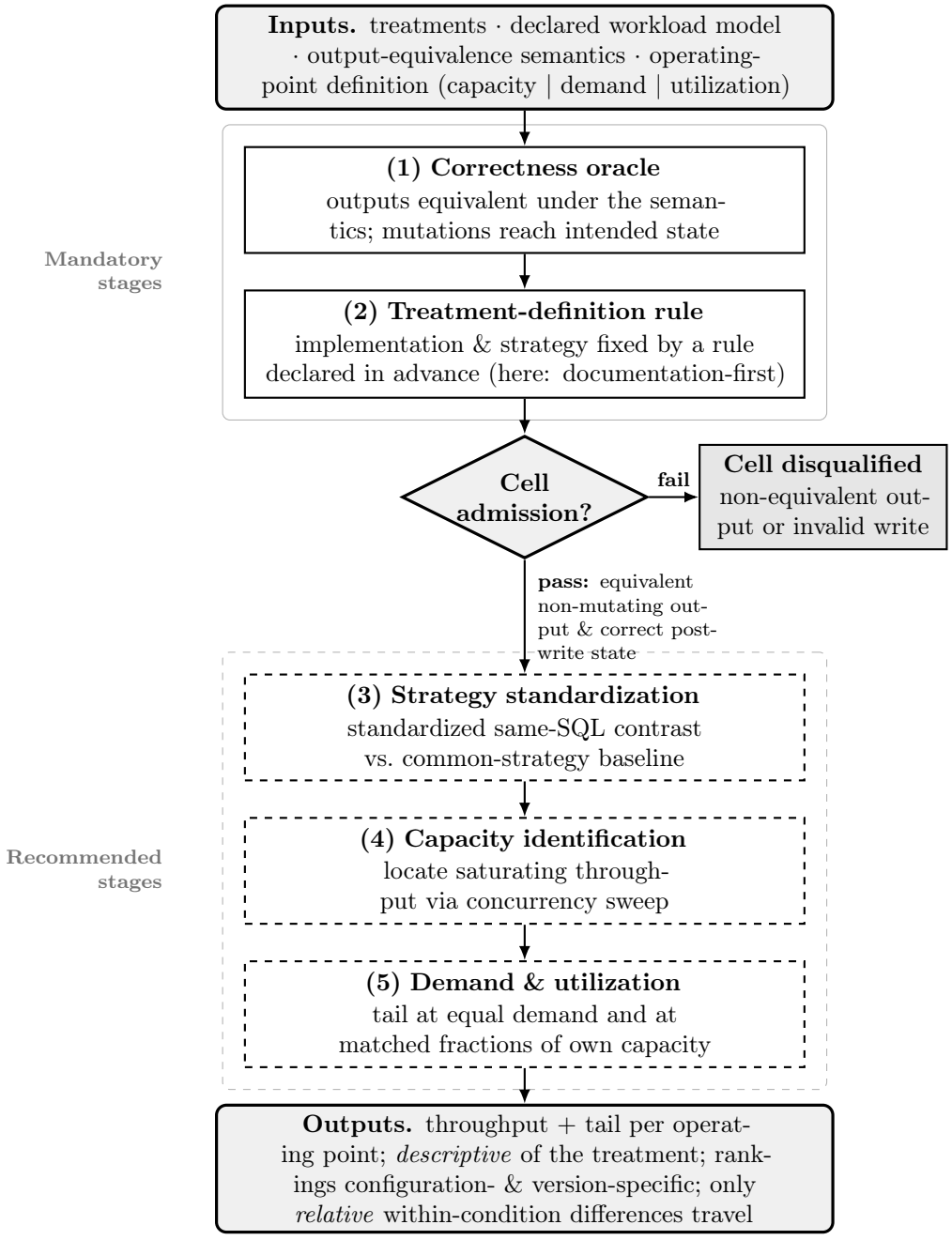

Figure 1: The comparability protocol as a flow. Declared *inputs* feed two *mandatory* stages (solid); the correctness oracle governs a per-cell *admission* gate—a (treatment, workload-point) cell is timed only if its non-mutating outputs are equivalent and its post-write state correct, while a non-equivalent output or an invalid write disqualifies it; query strategy (N+1, query count, joins, eager loading) is a reported characteristic, not an admission criterion, unless the workload explicitly forbids it. Admitted cells may traverse three *recommended* stages (dashed) that enrich but do not gate the comparison. *Outputs* are descriptive of the treatment as defined: throughput and tail per operating point, with configuration- and version-specific rankings and only relative within-condition differences meant to travel.

Admission versus diagnosis The admission gate turns on three concerns that must not be conflated. *Semantic correctness* always disqualifies: a non-mutating output not equivalent under the oracle, or a mutation that reaches the wrong state, excludes the cell. *Workload conformance* disqualifies only where the study *explicitly declares* a workload constraint (e.g. a round-trip bound) that the cell then violates. *Strategy characteristics* — query count, N+1, joins, eager versus lazy loading — are instead *reported*, not gated, being captured by server-side statement logging (here the per-endpoint counts of Supplement Table S2); this study declares no "no-N+1" or query-count constraint — the documentation-selected estimand's treatment *is* whatever the official documentation leads to — so no cell is excluded for its query strategy, and an N+1 plan would be admitted and reported like any other.

Applicability limits the oracle must match the output semantics: byte-identical comparison, used here, is valid only when equivalent outputs are deterministic and canonically serializable, and otherwise (unordered collections, floats, timestamps, nondeterministic identifiers) needs a semantic comparator (set equality, tolerance, canonicalization). The case study's outputs are deterministic and canonically constructed, so byte-equality is valid here.

Compliance levels An instantiation need not exercise every stage on every cell. Of the five *compliance levels*, the *mandatory validity core* and *capacity characterization* hold across all five patterns on both engines, while the *standardized-strategy*, *utilization-controlled*, and *resource-normalized* extensions are demonstrated on the deep fetch (widest-spread pattern tested) as a representative point, not full-matrix coverage (Supplement Table S40).

Table 2 pairs each generic benchmarking principle with its access-layer manifestation, the failure if the stage is skipped, and the evidence it was load-bearing *here*: each stage was consequential in this case study (its omission would have changed a conclusion), though one study cannot establish universal necessity. We found no prior access-layer benchmark applying all five together; Supplement Table S37 shows how each prior benchmark's conclusion becomes uninterpretable once its absent stages are named.

3.2 Factors and treatments

We vary three factors. The *access layer* has eleven levels across three tiers (schematic below): two native drivers with two engine-specific tuned baselines (`pg-tuned` reuses named prepared statements, `mysql2-tuned` the binary protocol via `execute()`), the query builder Knex, and six ORMs; the

Table 2: The comparability protocol, stage by stage. Each row pairs a generic benchmarking principle with its concrete manifestation for relational database access layers, the failure that follows if the stage is omitted, and the evidence that the stage was load-bearing in this study. We do not claim to originate these principles; in the sources searched we found no prior access-layer benchmark applying all five together. Throughputs abbreviated PG (PostgreSQL) and MySQL; "conns" denotes client connections.

Stage	**Benchmark decision**	**Generic principle**	**Access-layer manifestation**	**Failure if omitted**	**Evidence it was load-bearing**
Correctness oracle	Which treatments are timed at all	Validity precedes speed; never time incorrect results	Layers differ in output shape and mutation semantics; a "fast" cell may be a silent error doing less work, whereas engines return one canonical result	Broken/incomplete layers look fastest and top the ranking	Broken MikroORM MySQL write led the insert ranking until the write-state gate excluded it (silent 500s); a read cross-check caught a fan-out aggregation bug and a driver result-shape mismatch
Treatment-definition rule	Which API and loading strategy of each layer is measured	Pre-register and fix the system-under-test configuration reproducibly	One library exposes many equally-official query paths with different round-trip counts; a pure engine has one query language, so no selection rule is needed	Arbitrary/cherry-picked API choice; spread reflects the tester, not the defaults	Documentation-selected deep fetch spreads 7.15× (PG) / 4.96× (MySQL); round-trips differ 1–4 by design; Drizzle is the documented tie-break case
Strategy standardization	Add a standardized same-SQL contrast (a compound diagnostic, not a mechanism decomposition)	Hold confounds constant via a common-baseline contrast	Access-layer time mixes loading strategy with execution machinery; engines have no ORM loading strategy to confound, so no same-SQL baseline is meaningful	Mis-read the whole documentation-selected spread as intrinsic library "overhead"	The 7.15×/4.96× spread narrows to 1.68× (PG) / 2.01× (MySQL) among raw paths on common SQL, so most of the documentation-selected spread disappears under this compound raw-SQL standardization; which component is responsible is not identified
Capacity identification	Locate each layer's saturating throughput before fixing an operating point	Interpret latency relative to capacity, not at an arbitrary fixed load	Layers saturate at very different throughputs, so a fixed connection count lands them at different capacity fractions; a single engine saturates near one throughput	A fixed load compares layers at unequal utilization	A per-pattern sweep (1–200) puts every knee at or below 50 conns; median utilization 97–100% at the 50-conn point; native ∼7× the slowest ORM's capacity
Demand & utilization	Report tail at equal demand and at matched own-capacity fractions, as distinct quantities	Distinguish demand from utilization; correct coordinated omission	Because layer capacities differ, an equal-demand tail conflates queueing with sub-saturation latency; engines at similar capacity make the two nearly coincide	Read a capacity/queueing gap as a sub-saturation latency difference	At 50 conns deep-fetch p99 runs pg 20 ms to MikroORM 116 ms; at 50% of each layer's own capacity the tails converge to ∼2–5 ms, so the gap is capacity-and-queueing, not a sub-saturation latency difference

Table 3: Estimands defined in this study, consolidating the identification qualifications stated across Sections 3 and 4. Each quantity is *descriptive* of the treatment as defined; only relative within-condition differences are meant to travel, and rankings are configuration- and version-specific.

Estimand / quantity	**What it identifies**	**What it does *not* identify**	**Where reported**
Documentation-selected implementation-and-strategy effect (RQ1, primary)	Performance a developer following each library's documentation obtains (throughput, p99), as defined	Intrinsic library overhead; the most common or most performant usage	Tables 4, 5
Same-SQL (raw-path) standardized contrast (diagnostic)	Residual spread once every layer runs common SQL through its raw facility	A causal decomposition or a bound on the library effect; the share from SQL formulation or query count	Supplement Table S42
Performance-conscious deep-fetch regime	Speed recoverable within a narrow byte-equivalent tuning budget (faster documented loading strategy)	Gains from SQL rewrites, caching, or schema change; non-drop-in strategies (excluded)	Table 7
Per-pattern capacity (saturating throughput)	Each pattern's throughput knee per layer and engine, locating relative utilization at the operating point	Tail latency; a throughput quantity, not a demand- or utilization-dependent comparison on its own	Supplement Table S35, Figure S2
High-load p99 under equal closed-loop demand (primary tail)	The tail a deployment sees at fixed 50-connection demand, including acquisition queueing	The tail at comparable utilization; the pooled cross-run p99	Table 4
Matched-utilization tail (p99 at matched fractions of own capacity)	p99 when each layer is offered equal fractions (50/70/85/95%) of its *own* saturating throughput	The tail under equal external demand; the capacity-and-queueing component it removes	Supplement Table S43
Layer×engine interaction magnitude (RQ2)	The spread of each layer's PostgreSQL÷MySQL advantage and the concrete rank reversals	The interaction's size from the permutation test (bounds existence only); its sign (every layer faster on PostgreSQL)	Supplement Table S28; §4 (RQ2)
Cross-engine rank correlations ($n = 7$, coarse descriptive)	Descriptive agreement of the layer ordering across engines (Spearman ρ, Kendall τ)	Reliable cross-engine transfer at $n = 7$; the RQ2 finding, which rests on the reversals and interaction spread	§4 (RQ2)

tuned baselines are reference points, not documentation-selected treatments, leaving *nine* documentation-selected layers.

Tier	Implementations	Runs on
Native driver	`pg`, `mysql2`	its own engine
Tuned native baseline	`pg-tuned`, `mysql2-tuned`	its own engine
Query builder	Knex	both engines
ORM	Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM	both engines

The 7 portable layers run on both engines and the 4 native/tuned on one, so $4 + 7 \times 2 = 18$ layer×engine cells $\times$ 5 patterns $= 90$ measured configurations. We classify a library by its dominant interface, not its self-description (Section 1). The nine documentation-selected layers are a representative cross-section, not an exhaustive catalogue — each maintained, used, and (portable tiers) running on both engines — excluding non-portable PostgreSQL-only Slonik and pg-promise and query-builder Kysely (covered by Knex; `METHODOLOGY.md` gives the criteria). *Non-vendor authorship* means only that no evaluated library's maintainers were involved; the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed. The *engine* has two levels (PostgreSQL 18.4, MySQL 9.7.1) and the *access pattern* five, realized as HTTP endpoints (Supplement Table S39). Supplement Table S29 records which factors are held constant, vary, or are uncontrolled on this single virtualized host — the reason we report this configuration rather than general library performance.

3.3 Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1{-}*$ `posts` $1{-}*$ `comments` $*{-}1$ `authors`). The five endpoints implement five representative access patterns [18] (Supplement Table S39): a keyset page (primary-key cursor, not a large `OFFSET`), a single-row read, a one-to-many list, the deep/nested fetch (a post with its author and every comment, each with its own comment-author), and a per-author aggregation. A deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) is loaded by the native driver, independently of any layer under test, so both engines receive byte-identical row values; on-disk representation, index layout, and collation naturally differ. The working set fits in memory, so measurements emphasize the access layer's per-request instruction, protocol, and mapping cost, disk-read latency largely removed, while engine-side execution, buffering, locking, and logging remain in every measurement [26]. Each request's target identifier is drawn uniformly from

the seeded range (posts 1–100,000, authors 1–2,000), spreading load across the working set; a skewed, production-like distribution is a planned variant (Section 6).

3.4 The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the server and load runner are layer-agnostic and each returns an *identical result shape*. Each avoids the deep-fetch N+1 problem by its *documentation-selected* mechanism — a hand-written join for the native drivers, Knex, and query-builder-style Drizzle (tuned baselines reuse it), and relation eager loading (`include`, `populate`, `relations`, ...) for the data-mapper ORMs. The comparison is thus of each layer's *documentation-selected configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL generated, the round-trip count, and result materialization. Server-side logging captures the exact SQL, round-trip count, and `EXPLAIN` plans; per-endpoint counts differ by design (Supplement Table S2).

The documentation-selected API was fixed by a rule decided *before* measurement: the eager-loading API each layer's official documentation presents first for the pinned version, ties broken (as for Drizzle's two builders) toward the API at the library's own taxonomy tier. This is an intentionally artificial, predeclared *policy*, not evidence that the chosen API is performance-optimal or the most common production choice — one instance of the protocol's predeclared treatment rule (Section 3.1), which privileges none; a study targeting expert-tuned usage would substitute its own (corroboration and limits: Table S33, Section 6). Supplement Table S16, `METHODOLOGY.md`, and the rubric (`notes/documentation-selection.md`) record, per layer, that API (documentation page, version, URL), its round-trip count, the documented alternative not used, that logging, hooks, and validation are off, and the freeze date for reassessment against Wayback Machine snapshots.

To standardize the SQL and report how much spread remains once it is held fixed — the protocol's *strategy-standardization* precondition — every adapter also implements a *same-SQL standardized contrast*: the same two-statement deep-fetch SQL and row mapping (per engine, the same `EXPLAIN` plan) through the layer's raw-SQL facility, mirroring the aggregation control. Server-side capture, synchronized to a request marker so handshakes are excluded, confirms these statements are byte-identical across every layer on both engines; only the wire protocol differs (Supplement Table S14). Because the six ORMs split both ways on the deep fetch's join-versus-select-in default, that pattern — the one exposing a documented strategy choice — admits

two *co-primary* regimes reported side by side (Table 7): the *documentation-primary* regime fixed by the rule above and a *performance-conscious* regime applying a predefined *tuning budget*, the faster documented loading strategy admitted only where byte-identical to the documentation-primary output under the `bench/verify.mjs` oracle. The budget is deliberately narrow — only the loading strategy, only on the deep fetch, never rewriting SQL, adding caching, or changing the schema.

To ensure no adapter silently returns less, the *semantic-equivalence* precondition is established *before any timing* and at four levels rather than asserted as universal: shared canonical constructors fix each response's fields, key order, and serialization; a fast gate (`bench/verify.mjs`) asserts full JSON byte-equality to the native-driver baseline on 12 fixed probes per adapter on both engines; *exhaustive* equivalence is infeasible for arbitrary code and not claimed; and property-based randomized differential testing (`bench/verify-property.mjs`) asserts byte-equality over a fixed-seed sample of thousands of random and edge-case inputs on both engines, with *zero* divergences (Supplement Table S38) — so byte-equality stays a *finite*, if much broadened, check, strong evidence of, not proof of, agreement on every input. A companion check (`bench/verify-writes.mjs`) validates database *state*, not HTTP status: an independent native-driver connection confirms, for every adapter on both engines, that the primary single-row autocommit insert wrote exactly the requested values and identifier, and that the exploratory transactional write committed its post and five comments atomically, rolling back to no partial state when the last comment violates the author foreign key; all pass. The read cross-check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; Section 5).

3.5 Controls

Four controls hold the shared conditions fixed — connection pool, logical task, physical dataset state, and observer — so the *configured access-layer implementation bundle* is the only *deliberately varied* factor; they do not isolate the library alone: the bundled differences in SQL, query count, protocol, loading strategy, and hydration are the treatment (Supplement Table S42), not confounds to remove. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison reflects the access-layer choice, not pool tuning [25], [38] (Prisma's core-count-derived default is pinned via `connection_limit`). Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests always wait in each library's acquisition queue; this queueing is deliberately measured and held identical across layers (a 200 ms sampler confirmed the native PostgreSQL pool

fully occupied, about 32 requests pending). *(ii) Identical query semantics*: each endpoint issues the same logical query across layers (aggregation via one correlated-subquery formulation, the documented raw-SQL path), so the residual reflects the layer on an identical plan, not the SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, and the write endpoint is measured in its own dedicated boot after the physical state is rebuilt (recreated from a seeded template on PostgreSQL; rebuilt with `OPTIMIZE TABLE` and re-pinned `AUTO_INCREMENT` on MySQL), so physical layout and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.6 Measurement procedure

Load was generated with `autocannon` [16], a closed-loop (constant-concurrency) HTTP/1.1 generator [51] recording a latency histogram. Each of the 90 configurations was measured as 25 *repeated runs*: per replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up whose measurements were discarded — long enough to reach and sustain at least 95% of steady-state throughput, since managed runtimes do not always stabilize promptly [3] — then took one twelve-second measured run. Booting fresh isolates each replicate from prior JIT/pool/heap state; all runs share one host and one short campaign, so replicate index and measurement order are treated as *blocking factors*, and cell order was reshuffled independently within each replicate (a forward-versus-reversed check found no history effect on the read cells). Within a replicate, every layer and engine is driven by the identical request-id sequence from a PRNG seeded on the (endpoint, replicate) pair, so this *paired-stream* design removes between-layer sampling noise as a confound. The experimental unit is one freshly booted (layer, engine, endpoint) run per replicate; individual requests are subsamples. We report the median of the 25 replicate values of throughput and of the run-level percentiles p50, p90, p97.5, and p99 (one cell has 24 after a failed health check; the median and interval accommodate it). Throughout, *p99* denotes this *median run-level p99* — the median across replicates of each run's own p99 — not the p99 of the pooled request distribution, which the design does not estimate. A 60 s re-measurement of the deep fetch preserves the p99 ranking and barely moves each cell's p99 (Supplement Table S23), so the 12 s window suffices for the tail. We sampled server CPU and peak RSS over the process tree, with database and load-generator CPU separately. Inserts were

measured under each engine's *default* durability (PostgreSQL `fsync` and `synchronous_commit` on; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`), the regime a deployment runs unless relaxed; a labelled secondary run relaxed all of these to isolate the durability mechanism (Supplement Table S3). To characterize behavior under load we swept the deep/nested fetch across 1–256 connections for every layer and engine [61]; and a native `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring from intended send time and clipping timeouts at their cutoff — a coordinated-omission-corrected design [35], [54] that independently validates the closed-loop tail (Section 4).

3.7 Analysis

We separate *primary* outcomes, which carry the research-question claims, from *secondary* and *exploratory* outcomes reported descriptively as controls and sensitivity analyses (Supplement Table S34): the primary outcomes are throughput and the closed-loop p99 on the five patterns against both engines at the fixed operating point.

Because absolute throughput is hardware-specific, we analyze *relative* differences within an engine; a pattern's *spread* is the documentation-selected native driver's median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns). We report medians over the 25 runs, the coefficient of variation as a stability signal [24], [37], and a fixed-seed percentile bootstrap 95% CI for the median; these intervals capture within-campaign, run-to-run variability on this host and configuration, not variation across machines, versions, or deployments (Section 6). The design is a *randomized block* — within each replicate every layer runs the identical request stream — so adjacently ranked layers are compared with *paired* tests on their per-replicate throughput ratios (a two-sided sign-flip permutation test cross-checked with Wilcoxon signed-rank), reporting the geometric-mean paired ratio, its bootstrap CI, the paired dominance, and a TOST against a $\pm 5\%$ margin (author-selected and application-dependent, not universal; supplement) for the closest pair; we foreground these effect sizes and interval estimates over p-values [1], [13]. Because the seven adjacent pairs follow the *observed* ranking, their significances are a post-hoc, descriptive robustness check, not a confirmatory family. Adjacent-layer p99 gaps of one or two milliseconds sit at the millisecond measurement floor and are not read as real; conclusions rest on the large-ratio patterns, chiefly the deep fetch. The full construction (procedure and seed, tie handling, the layer$\times$engine interaction test, rank correlations, equivalence margin) is in the

statistical-methods notes (`notes/supplement-methods`); paired p99 comparisons are Supplement Table S30.

3.8 Experimental setup

Measurements were taken 2026-07-16 to 2026-07-18 on a single host — a virtualized Intel Xeon Gold 6140 instance (16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), PostgreSQL 18.4 and MySQL 9.7.1 local — the primary matrix overnight and fourteen labelled secondary experiments (order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, concurrency, utilization, pool-size, cluster, mixed-workload, wait-event, post-restart, longer-window tail) over the same window. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); only Sequelize invokes the earlier-release exception (its version-7 line is a prerelease, so the current 6.x stable line is used). Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking (Section 4).

3.9 Replication package

The harness — the Express application, the adapter contract, all eleven adapters, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts regenerating every table in Section 4 — is released so the full matrix re-runs with one command against a containerized or local PostgreSQL and MySQL.

4 Results

Tables report throughput (req/s, higher better) and tail latency (p99 ms, lower better) per access layer and engine. The two consequential patterns are in the main text (deep/nested fetch, Table 4; insert, Table 5); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15, all from the run of Section 3. Each native driver appears documentation-selected (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking the hand-tuned ceiling, not one attained without code changes. We compare *relative* differences within an engine, where two estimands recur: the *documentation-selected implementation-and-strategy effect* (RQ1–RQ3), what each library's documentation-selected configuration

yields rather than its intrinsic overhead; and the *same-SQL effect* (Supplement Table S14), what remains when every layer runs common SQL through its raw facility — a compound standardized contrast reporting the residual spread, not decomposing or bounding the difference (construct validity: Section 6).

Table 4: Deep/nested fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3687 [3602–3737]	20 [18–21]	–	–
mysql2	native-driver	–	–	2382 [2357–2451]	28 [28–29]
pg-tuned	native-tuned	3746 [3684–3828]	18 [17–18]	–	–
mysql2-tuned	native-tuned	–	–	2415 [2387–2427]	25 [24–26]
knex	query-builder	2487 [2465–2505]	27 [26–28]	2007 [1989–2031]	30 [29–31]
drizzle	orm	1865 [1837–1878]	35 [32–35]	1318 [1300–1336]	47 [46–48]
prisma	orm	1186 [1160–1203]	51 [51–53]	634 [618–644]	93 [89–95]
sequelize	orm	991 [977–998]	60 [59–61]	886 [868–898]	67 [66–68]
typeorm	orm	1204 [1188–1223]	50 [49–52]	1017 [987–1029]	57 [56–59]
objection	orm	1028 [1020–1037]	57 [56–58]	834 [828–855]	69 [67–71]
mikroorm	orm	516 [499–525]	116 [113–119]	480 [474–488]	120 [118–128]

4.1 RQ1: performance difference relative to the native baseline

The native driver leads all ten engine-pattern combinations: `pg` (or its tuned counterpart) is fastest on all five PostgreSQL patterns, and `mysql2` on all five MySQL ones. The current-generation ORMs sit mid-to-low throughout, Prisma among the lowest — near the bottom of both point reads and aggregation on both engines, and last on the MySQL insert (Tables 4, 5; Supplement Tables S12 and S15) — retiring an earlier version's headline of Prisma tying the native driver at a large CPU premium (Section 5).

`pg-tuned` and `mysql2-tuned` track their documentation-selected counterparts within a few percent except on the deep fetch, where a second round trip lets prepared statements pay off (`pg-tuned` 3,746 vs. `pg` 3,687; `mysql2-tuned` 2,415 vs. `mysql2` 2,382); on PostgreSQL aggregation `pg-tuned` adds about 8% (7,538 vs. 6,978).

Table 5: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6402 [6242–6547]	11 [10–15]	–	–
mysql2	native-driver	–	–	1753 [1717–1763]	113 [109–119]
pg-tuned	native-tuned	6413 [6073–6571]	11 [10–12]	–	–
mysql2-tuned	native-tuned	–	–	1750 [1482–1759]	120 [111–130]
knex	query-builder	4841 [4629–4908]	15 [14–16]	1670 [1470–1692]	127 [117–137]
drizzle	orm	4085 [3928–4146]	15 [14–18]	1730 [1324–1745]	120 [109–138]
prisma	orm	2774 [2750–2815]	23 [22–24]	886 [828–903]	153 [137–160]
sequelize	orm	2732 [2719–2747]	23 [21–25]	1478 [1375–1684]	125 [118–132]
typeorm	orm	2648 [2630–2694]	23 [22–26]	1345 [1301–1359]	125 [119–129]
objection	orm	3907 [3746–3980]	17 [15–23]	1728 [1598–1746]	117 [108–132]
mikroorm	orm	1979 [1949–1997]	30 [29–31]	1257 [1240–1268]	117 [109–125]

Table 6: Analysis roster. The *primary* analyses carry the confirmatory, paired inference and the research-question claims; the *robustness / sensitivity* analyses control or contextualize them. Main-text tables are cross-referenced; supplement tables are cited by their *S*-number.

Analysis	Role	Location
Deep/nested-fetch throughput & closed-loop p99	Primary	Table 4
Insert (write) throughput & p99	Primary	Table 5
Point read, range scan, aggregation: throughput & p99	Primary	Supplement Tables S12, S13, S15
Predefined native-driver reference contrasts	Primary	Supplement Table S41
Deep-fetch regimes (documentation-primary, performance-conscious)	Primary	Table 7
Same-SQL standardized contrast	Robustness / sensitivity	Supplement Table S42
Matched-utilization / equal-demand tail	Robustness / sensitivity	Supplement Table S43
Per-pattern capacity / utilization sweep	Robustness / sensitivity	Supplement Table S35
Equal-CPU budget (exploratory)	Robustness / sensitivity	Supplement Table S4
Pool-size frontier (1–50)	Robustness / sensitivity	Supplement Tables S7, S25
Multi-worker cluster scaling	Robustness / sensitivity	Supplement Table S24
Open-loop coordinated-omission tail	Robustness / sensitivity	Supplement Tables S6, S17
Longer-window (60 s) tail sensitivity	Robustness / sensitivity	Supplement Table S23
Property-based semantic-equivalence sweep	Robustness / sensitivity	Supplement Table S38
Layer × engine interaction magnitude	Robustness / sensitivity	Supplement Table S28
Paired significance ladders (throughput, p99)	Robustness / sensitivity	Supplement Tables S36, S30

On the **point read** the native-relative spread is 2.78× (PostgreSQL) and 2.11× (MySQL) (Supplement Table S12). On the **deep/nested fetch** (a post, its author, and all comments with their authors) it is the widest measured — 7.15× on PostgreSQL (`pg` 3,687, 95% CI [3,602–3,737], down to MikroORM's 516 [499–525]) and 4.96× on MySQL (`mysql2` 2,382 [2,357–2,451] down to MikroORM's 480 [474–488]). The native driver leads on both engines, the data-mapper ORMs (especially MikroORM) trailing furthest (Table 4); the comparison is paired (Section 3), every adjacent MySQL step large relative to its bootstrap interval (Supplement Table S36). **Aggregation** is the least layer-sensitive pattern: the native-relative spread is only 1.83× (PostgreSQL) and 1.72× (MySQL) (Supplement Table S15) — small because each layer forwards a single query, unlike the deep fetch's application-side materialization of a nested object graph.

4.2 Same-SQL standardized contrast

The *same-SQL standardized contrast* (Section 3) is a *compound* intervention: replacing each layer's documentation-selected loading path with the same hand-written SQL through its raw facility changes several mechanisms at once (Supplement Table S42). Server-side capture confirms byte-identical statement text across layers on both engines, though wire protocols differ per layer, so the contrast is *same SQL, protocol disclosed*, not *same plan*. Because those mechanisms may interact differently with each library, this residual is a diagnostic contrast, *not* a bound on the intrinsic library effect.

Among the raw paths the deep-fetch native-relative spread is far smaller — 1.68× (PostgreSQL) and 2.01× (MySQL) against the documentation-selected 7.15× and 4.96× (Supplement Table S14). This supports only that the documentation-selected paths differ much more than the raw paths; it does *not* quantify how much of the original difference comes from SQL formulation or query count, which change together with the API, protocol, preparation, and hydration (Supplement Table S42) — attributing the residual to any one needs a factorial or staged ablation (future work). A loading-strategy sensitivity check (Supplement Table S18) argues against a mere default-strategy artifact: forcing the join-default ORMs (Sequelize, MikroORM) to select-in makes them slower, and Objection's batched-select-in default runs about 1.6× slower than a single join on MySQL, so part of *its* deficit is a strategy choice, not a fixed cost.

Because the deep fetch is the only pattern exposing a documented strategy choice, we report RQ1 there under *two co-primary regimes* (Section 3): the documentation-primary strategy and a performance-conscious one taking the faster byte-identical documented strategy (Table 7). It shifts little:

the join-default ORMs (Sequelize, MikroORM) already use the faster strategy, so both regimes give the *identical* configuration, and only Objection on MySQL gains — its single-join alternative beats select-in, while TypeORM's PostgreSQL alternative errors and Objection's breaks byte-identity, both excluded as non-drop-ins. The performance-conscious ORM deep fetch still trails the native driver, so the gap reflects the libraries' documented strategies, not a poor-default artifact, and this regime caps how much a developer could recover within a narrow, semantically-equivalent tuning budget.

Table 7: The two co-primary deep-fetch regimes, measured on one harness (fixed post id, a warmup pass then 25 timed repeats each; absolute req/s therefore differ from the seeded-id primary matrix by design, but every cell here shares the harness, including the native reference row). *doc-primary* is each layer's documentation-selected loading strategy; *perf-consc.* is the performance-conscious regime — the faster of the layer's two documented deep-fetch strategies (join versus select-in), admitted only where byte-identical to the documentation-primary output. For Sequelize and MikroORM the documentation-selected join is already the faster strategy, so the two regimes coincide; only Objection on MySQL gains (its documented single-join alternative beats its select-in default), and even then its performance-conscious throughput stays below the native driver. A dash marks a layer×engine cell whose documented alternative is not a byte-identical drop-in in the pinned versions (typeorm/postgres (alt endpoint status 500); objection/postgres (response not byte-identical); typeorm/mysql (alt endpoint status 500)); there the performance-conscious regime coincides with documentation-primary by definition.

		PostgreSQL (req/s)		MySQL (req/s)	
Layer	Documented alt.	doc-primary	perf-consc.	doc-primary	perf-consc.
native (`pg`/`mysql2`)	hand-join (single)	3347	—	2274	—
`sequelize`	join→select-in	894	894	818	818
`mikroorm`	join→select-in	409	409	416	416
`objection`	select-in→join	–	–	801	1262
`typeorm`	join→select-in	–	–	–	–

4.3 RQ2: cross-engine transfer of the ranking

The read ordering is stable across engines, but the aggregation and insert orderings reverse at the top, so we characterize transfer by these rank rever-

sals and layer×engine interaction magnitudes, not a correlation over seven portable layers. On the **insert** `knex` leads on PostgreSQL (4,841 req/s) but drops to third on MySQL (1,670), where `drizzle` takes the lead (1,730); Prisma falls furthest, from fourth on PostgreSQL (2,774) to last on MySQL (886, behind even MikroORM's 1,257), a reversal invisible to a single-engine benchmark. **Aggregation** repeats the shape: `drizzle` leads on PostgreSQL (6,280) but falls to fourth on MySQL (3,520), where `knex` leads (3,842), while Prisma and Objection swap the bottom two (Supplement Table S27). In *magnitude* (Supplement Table S28, per-layer PostgreSQL÷MySQL ratios) every layer is faster on PostgreSQL, so the interaction lies in the *spread*, not the sign: across the three reads it stays within ≈1.0–1.9×, but the insert scatters from 1.6× (MikroORM) to 3.2× (Prisma) and aggregation from 1.4× to 1.9× — a two-to-one range on the writes that reorders the layers. A blocked permutation test (Section 3) confirms the interaction is nonzero but bounds only its existence; the coarse ($n = 7$) rank correlations (Supplement Table S27) are descriptive only. Because *engine* here varies the whole stack — driver, wire protocol, adapter, engine implementation, and default isolation and durability — together (for Prisma, even a different MariaDB adapter on MySQL; Section 6), these are differences between the tested PostgreSQL and MySQL *stacks*, not an effect of the database engine alone. Single-engine results are thus portable in order for the reads, not the writes.

On the **keyset range scan** the native driver again leads (native-relative spread 4.21× on PostgreSQL, 3.85× on MySQL; Supplement Table S13), driven disproportionately by MikroORM alone (905 / 856 req/s); the earlier collapse under `OFFSET` pagination was a workload artifact, not an engine or layer property (Section 5).

Inserts are where the engine dominates most visibly, under each engine's *default* durability configuration (Section 3). On MySQL the faster layers cluster within about 5% near an engine-imposed floor (`mysql2` 1,753 down to Knex 1,670 req/s; Table 5), Prisma the exception at 886 (last), widening the nominal MySQL spread to 1.98×. Direct `performance_schema` instrumentation attributes the floor to the commit path: relaxing durability lifts the flush-bound native driver and query builder about 2× while the slower ORMs, limited by their own write overhead, gain less (Supplement Tables S3, S19), so the commit path, not the layer, is the floor for the lean layers. PostgreSQL, by contrast, spreads 3.23× (`pg` 6,402 down to MikroORM 1,979) and is only weakly durability-sensitive (≤14%), so there the access layer, not the engine, remains the larger factor.

4.4 RQ3: which pattern spreads widest

Ranking the five patterns by native-relative spread gives, on PostgreSQL, deep fetch (7.15×) > range scan (4.21×) > insert (3.23×) > point read (2.78×) > aggregation (1.83×); MySQL keeps the same head and tail (deep fetch 4.96×, aggregation 1.72× last) while the insert and point read swap in the middle, the insert compressing because MySQL's engine floor, not the access layer, sets its ceiling (RQ2). The deep/nested fetch thus has the largest native-relative spread among the five tested patterns on both engines, aggregation the smallest. A fixed 50-connection demand could place patterns at different capacity fractions; a per-pattern concurrency sweep (Supplement Table S35) rules this out, every pattern's throughput knee at or below 50 connections on both engines and all five at high utilization of their own capacity (median 97–100% across layers). The primary deep-fetch workload uses posts with roughly ten comments; a fan-out sweep from 0 to 500 comments shows the native-relative spread is a roughly fixed multiplier with graph size rather than growing with it (exploratory; an earlier growing-spread finding did not replicate).

4.5 Tail latency

We report high-load p99 under equal closed-loop demand at the 50-connection point, so the tail includes connection-acquisition queueing — what a deployment sees under load. Here tail latency tracks throughput: on the deep/nested fetch PostgreSQL's p99 ladder runs from `pg` 20 to MikroORM 116 ms (Table 4); a paired p99 analysis (Supplement Table S30) resolves every adjacent pair on both engines except two near-ties at the millisecond resolution. Because each 12 s run completes roughly 6,000 (slowest ORM) to 44,000 (native) deep-fetch requests, a run's top-1% p99 rests on only ∼60–440 observations and is noisier for slower layers, so we report the median of the 25 run-level values rather than pooling requests (spread in Supplement Figure S4). Offering each layer 50/70/85/95% of *its own* saturating throughput (Supplement Tables S21–S22) the tails converge (near 2–5 ms at 50% utilization on both engines), so the large high-load gap (`pg` 20 ms versus MikroORM 116 ms) largely dissolves at matched utilization: it is a capacity-and-queueing effect, not a difference in latency at a matched offered-load fraction. The coordinated-omission-corrected open-loop harness (Supplement Tables S6, S17) confirms this, corrected tails flat below saturation and collapsing beyond capacity; Supplement Table S43 shows both readings per layer, averting the false "worse sub-saturation latency" reading.

4.6 Measurement stability and effect sizes

Across the 25 repeated runs the coefficient of variation of deep-fetch throughput is at most 4.0% on either engine (Supplement Tables S1, S9), well below the between-layer spreads; dispersion is larger only on the insert, whose MySQL cells are bimodal under group-commit batching — a structure the coefficient of variation (up to 15.9%) cannot convey — so we read it from the replicate plot (Supplement Figure S1) and the insert-table bootstrap spread, reporting the insert rank correlation conservatively. Every measured request succeeded (zero errors or non-2xx across all 90 cells) — a load-bearing check, since a cell erroring faster than it works can spuriously lead a ranking (Section 5). We foreground paired effect sizes: the confirmatory contrast is *predefined*, not selected from the ranking — against the native-driver reference fixed before analysis, every portable layer trails it in all 25 replicates (dominance 1.00; Supplement Table S41), and adjacent ranks separate similarly, all but the narrowest pair (Sequelize over Objection, 1.05) exceeding the run-to-run CV. These bootstrap intervals are within-campaign, not deployment-general (Section 6); the adjacent-pair permutation and Wilcoxon tests, which re-test only pairs the ranking selected, are a descriptive supplement check (Supplement Table S36).

4.7 Scaling with concurrency

A concurrency sweep from 1 to 256 connections (Supplement Figure S2) shows the three-band ordering (native driver; then Knex and Drizzle; then the data-mapper ORMs) emerging by 8 connections and holding above about 32, with layers far closer at a single connection (about 320–1,020 req/s). The 50-connection deep-fetch point sits above every layer's knee, so the RQ1 ranking is not a single-point artifact.

4.8 Resource footprint

On the current library generation the application-side CPU cost is uniform (Supplement Tables S5, S10; trade-off plotted in Supplement Figure S3): every layer draws about one core on the deep fetch (100–110%), so none buys throughput that way. The native driver also leads end-to-end compute efficiency (`mysql2` 1,254 versus Prisma 384 and MikroORM 356 req per combined app-plus-database CPU-second on the MySQL deep fetch; Supplement Table S5). These CPU shares, measured at unequal throughput, describe efficiency but do not by themselves locate where per-request work occurs. Exploratory equal-CPU and 1-to-4-worker cluster checks (Supplement Tables S4,

S24) suggest low per-process throughput is met by adding application processes until another bottleneck, with no database-side saturation measured. Peak memory ranges 215–356 MB, lowest for the query builder and lightest ORMs (Supplement Table S10) [57].

5 Discussion

Version-sensitivity An earlier campaign found Prisma (then 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores; the *identical* harness on the latest stable releases did not reproduce it — no layer now draws over one core, and Prisma sits at mid-to-low throughput (Section 4). The non-reproduction is a fact, its cause a hypothesis (Prisma's Rust-free rewrite is likeliest, though the re-freeze moved the whole toolchain at once), so the headline is version-sensitive [45], [64].

Implications for practice RQ1 and RQ3 agree the documentation-selected implementation-and-strategy difference is specific to the access *pattern*: the native-relative spread is widest on the deep/nested fetch — where a layer must hydrate rows and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [30] made concrete) — and narrowest on aggregation. Round-trip count alone does not set throughput — result-graph materialization does: on the PostgreSQL deep fetch single-query MikroORM is slowest while two-query Drizzle outruns four-query Prisma (Supplement Table S2; Table 4).

Compute cost and horizontal scaling With every layer now near one core, the native driver leads throughput and compute efficiency (requests per CPU-second; Section 4, Supplement Figure S3, Table S5), consistent with its leaner SQL — though CPU shares at unequal throughput do not identify where per-request work occurs. Low per-process throughput is no fixed ceiling: on PostgreSQL Prisma's aggregate throughput rises roughly with workers (1,117 to 4,449 over one to four) to native-like levels, trading footprint and energy for it [5], but this bounded 1-to-4-worker check on a single host is not general scalability: on MySQL the native driver keeps scaling, the gap not closing and no database-side saturation located. The open-loop cross-run further shows a closed-loop benchmark would underestimate how abruptly, not just how much, a heavyweight ORM's tail degrades once load crosses its capacity.

Single-row inserts: a partly engine-bound cost under default durability RQ2 has a clear answer on single-row inserts, under each engine's *default* durability: the native driver leads on both, PostgreSQL's inserts staying strongly layer-dependent even with fsync on (3.23×; Table 5), whereas on MySQL the faster layers cluster within about 5% near an engine-level commit floor (the per-commit redo-log-then-binlog double flush [26]; Supplement Table S19); relaxing durability does not overturn this contrast. The ranking reverses across engines — `knex` leads on PostgreSQL, `drizzle` on MySQL, Prisma dropping from fourth to last (Table 5) — the per-layer engine advantage spanning 1.6–3.2× (Supplement Table S28), so a single-engine benchmark like [50] misses this interaction either way, which the dual-engine design exposes, as prior work on database technology modulating mapping cost anticipates [43]. An *exploratory* transactional multi-statement write (a post and five comments; five replicates, representative layers) narrows the spread to about 3.7× (Supplement Table S8), below the widest read spread, because one commit amortizes the flush across six inserts; Prisma, mid-pack on the single-row insert, falls to the bottom transactionally, so the ordering does not track the single-row one.

Methodological lessons: a checklist for access-layer benchmarking Four confounds would otherwise have silently corrupted the comparison; we distil them into a reusable checklist (`notes/supplement-methods`): an **aggregation fan-out** (a join before `SUM` inflating the aggregate) and a driver result-shape mismatch, both *correctness* bugs the automated byte-equality cross-check caught; **OFFSET pagination** collapsing on MySQL, easily misread as an access-layer weakness; **write-workload contamination** from an unreset growing table; and a **query-formulation confound** where a mis-scoped pre-aggregation re-scanned the whole table, a query-plan artifact masquerading as an access-layer effect [39]. The write path needs its own gate: MikroORM 7's dropped `persistAndFlush` made every insert throw a 500 faster than a real one, briefly *leading* MySQL until the non-2xx counter flagged it. Because a 2xx alone would miss a *silently* incorrect write, the gate also validates post-write database state — field values, identifiers, row-count changes, and rollback (Section 3) — the write path's correctness gate as byte-equality is the read path's. None is schema-specific; each extends the database-benchmarking pitfalls of Fruth et al. [22] and Raasveldt et al. [47] to the access-layer sub-genre, and at least one plausibly explains part of the gap between some vendor-reported and independently verified figures.

Practical guidance The synthesis follows from RQ1–RQ3 and describes the benchmarked implementations in this configuration and workload — single host, default durability, pinned versions (Section 6) — not access-layer categories in general. Because the fan-out sweep (Section 4) varies breadth (0–500 children) but not depth, schema, or query mix, the documentation-selected spread's peak on the deep/nested fetch locates a difference among five probes on one synthetic schema — a benchmark observation to re-measure on the target workload, not an established law. Where a hot path is dominated by deep or nested fetches the access layer is consequential and worth benchmarking: the thin paths (native driver, Knex) led here *for throughput and response-time p99*. Where the workload is read-simple or write-bound (especially on MySQL, whose commit floor dominates under default durability) the cost is small and may be met by adding application processes until another bottleneck (contention, connection limits). These are performance findings only: the choice also turns on correctness, maintainability, security, type safety, and developer productivity, which this study does not measure and may outweigh them; and the same-SQL results are a standardized diagnostic contrast on the documentation-selected cost, not a licence to use raw SQL.

6 Threats to Validity

We organize the threats along the four standard validity categories [17], [58].

Construct validity of the measures Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip (Express routing, serialization), though the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We hold those near-constant: every adapter satisfies a documented *adapter contract* and funnels rows through shared canonical constructors (fixed field set, key order, typing, under `TZ=UTC`), and the automated cross-check (`bench/verify.mjs`) asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run (Section 3). That check caught a defect: PostgreSQL's timezone-naive `created_at` (the column is `timestamptz`) shifted Drizzle's instants by the host's UTC offset, invisible to throughput and fixed before this campaign. The fixed-payload baseline floor sits above the fastest cell (Section 3), so the framework compresses rather than manufactures between-layer differences, attributable to how each layer reaches an identical response,

not to what is returned. The reported write finding uses each engine's *default* vendor-standard durability (Section 3), not one we relaxed, so the write spreads (Table 5) describe a standard deployment, not a tuning choice; a symmetric relaxed-durability secondary confirms the cross-engine gap is not an artifact of either configuration (Section 5, Supplement Table S3).

Construct validity of the treatment RQ1's treatment is each layer's *documentation-selected* implementation and eager-loading strategy: an operational rule — the API the official documentation presents first — not validated against surveyed, typical, or performance-tuned usage. It is an intentionally artificial, predeclared selection policy, not a validated practitioner persona; where it differs from what a performance-conscious developer would choose, RQ1's spread mixes the library's execution machinery with that strategy choice, made material by the one-to-four statement range on the deep fetch (Supplement Table S2). Three things constrain it. First, the selected API is never obscure — for the ORMs it is the canonical eager-loading API (Supplement Table S33, corroborated per layer in `METHODOLOGY.md`). Second, the *same-SQL standardized contrast* runs common SQL through every layer's raw facility, so the raw-path spread (deep fetch 1.68×, against 7.15× documentation-selected; Section 4) is far smaller — a compound *standardized contrast*, not a decomposition or bound on how much comes from the loading strategy versus the library machinery. Third, where the documented alternative is a byte-identical drop-in, a loading-strategy check (Supplement Table S18) shows the documentation-selected default is the *faster* strategy for the join-default ORMs, so their deficit is not an artifact of a poor default; the lone exception, Objection's slower select-in default on MySQL, is disclosed rather than corrected. RQ1 therefore answers what this documentation-selected policy yields, not the library's intrinsic overhead.

Internal validity A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost, so every adapter uses its layer's documentation-selected eager-loading strategy on the deep-fetch endpoint (Section 3), and the byte-level cross-check caught two defects before measurement (Section 5). Per-cell warm-up phases exceeding the slowest layer's cold-start stabilization absorb JIT, pool-fill, and plan-cache effects [44]; two further controls rule out non-layer confounds — benchmark rows reset before every cell (physically rebuilt for writes), and one correlated-subquery aggregation plan across layers. Within a replicate every layer and engine runs the identical PRNG-seeded id sequence, and a forward-versus-reversed cell-order check found no history effect (Section 3), so order does

not confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [22], [54]; a native, coordinated-omission-corrected constant-arrival cross-run on the deep fetch (Section 3) is consistent with the tail ranking holding under that stricter model (Section 4; Supplement Table S6).

External validity All measurements come from a single schema, dataset size, and 16-vCPU virtualized host, a pool fixed at ten, and specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; latest harness-compatible stable, Supplement Table S11). The fixed pool does not drive the ranking: the pool-size frontier (Supplement Tables S7, S25) preserves the ordering from 1 to 50 connections, and a mixed read/write workload (S26) preserves the read ordering. Two choices are disclosed rather than resolved: Sequelize on its stable 6.x line (7.x is prerelease), and Prisma's MySQL path on the MariaDB adapter, as Prisma ships no `mysql2` adapter — an implementation×engine asymmetry. The memory-resident working set is a hot-cache regime that makes access-layer overhead more visible; as a workload becomes I/O-bound the spreads should compress toward 1×. Same-host checks bound only within-host drift, not an independent substrate: equal-CPU pinning agrees with the shared-host medians within about 2%, low-per-process layers scale roughly linearly over one to four workers (Supplement Table S24), a ten-minute run moves throughput by at most ±1.4%, and a post-restart re-run moved throughput up to 16% with the ranking intact (Supplement Table S20). Only the *relative* within-engine ordering and operating-point separation are intended to travel; absolute req/s, gap magnitudes, and RQ2's cross-engine ordering may move with the substrate and need not transfer to other working sets, pool sizes, or versions, so a substrate change should preserve these *architectural* conclusions while independent-host replication of the core deep fetch remains future work. Because the protocol's stages are experimental-design controls (Table 2, Supplement Table S37), single-host measurement bounds the case study, not the protocol.

Conclusion validity Our analysis (Section 3) reports medians with the coefficient of variation and within-campaign bootstrap 95% intervals (repeatability, not population-general) and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with Wilcoxon; Supplement Table S36). Two bounds reinforce the ordering: the widest spreads dwarf the run-to-run CV ($\leq 4.0\%$; Supplement Tables S1, S9), and it holds across the concurrency sweep at ≥ 32 connections (Supplement

Figure S2). Rather than assert a specific statistical power (absent a prespecified effect-size model), we rest the conclusions on effect sizes and interval estimates — foremost the *predefined* native-driver contrasts (Supplement Table S41), keeping the ranking-selected adjacent-pair permutation tests only as a descriptive post-hoc check. Booting a fresh process per replicate prevents persistent state carrying across replicates [32], but they share one host and one campaign, so we treat replicate index and measurement order as blocks rather than claiming full independence. The secondary experiments covering only a layer subset or few replicates (open-loop tail, equal-CPU, pool-size, transactional write, and cluster checks; Supplement Table S32) are read as *sensitivity evidence* on the tested subset, not as claims about all eleven implementations.

7 Conclusion and Future Work

This paper's contribution is a **comparability protocol** that turns access-layer benchmarking into a controlled experiment rather than vendor-style number-reporting: it admits a layer's timing only once that layer returns byte-identical results (semantic equivalence), pairs a documentation-selected estimand with a same-SQL standardized contrast (strategy standardization), and separates capacity, tail-under-demand, and matched-utilization latency (operating-point separation). An open, reusable harness operationalizes it, demonstrated by a dual-engine case study — eleven implementations across PostgreSQL and MySQL over five access patterns (Section 1).

The documentation-selected implementation-and-strategy difference is concentrated, not uniform, across the five tested patterns: sharpest for the deep/nested fetch assembling a nested object graph application-side, modest for point reads and aggregation. Among the raw paths running common SQL the spread is far smaller — a standardized diagnostic, not a bound (Section 4).

On current library versions throughput never scales with added application cores, and the native driver leads all ten engine-pattern combinations; read orderings are similar across engines while aggregation and insert orderings reverse (Prisma drops from mid-pack to last on the MySQL insert), cautioning against generalizing single-engine results. Rankings are also version-sensitive: an earlier headline did not survive re-running on newer releases (Section 5).

Development also surfaced a reusable checklist of genre-specific benchmarking pitfalls — one, a fast-but-wrong write path that passed the read-level cross-check, added by this very re-run (Section 5).

These findings held under a concurrency sweep (1 to 256 connections), resting on large between-layer effect sizes with non-overlapping bootstrap intervals, the descriptive adjacent-rank permutation tests agreeing (Section 4).

Extensions would deepen the evidence — a two-machine cross-run to bound observer interference beyond the open-loop check, more engines and dataset sizes, and a dedicated study of the N+1 penalty and cold-start latency — and the released harness, re-run against Prisma 7, makes these straightforward, addressing the repeatability gap documented in computer-systems research [14].

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table and figure) is available at `https://github.com/mmiotk/express-db-access-performance` release v1.12.11 is permanently archived at Zenodo (`https://doi.org/10.5281/zenodo.2149796` A reproducibility summary — versions, requirements, run commands, time and resources, and which results the harness regenerates automatically from the archived raw data — is in Supplement Table S31 and `REPRODUCE.md`. **Exact reproduction requires the reference engines** (PostgreSQL 18.4 and MySQL 9.7.1), installed via the user-space conda path documented in `REPRODUCE.md`; the convenience Docker path pins *older* engines (PostgreSQL 16, MySQL 8.4) and reproduces the workflow but not the published numbers. A machine-readable encoding of the protocol — inputs, mandatory and recommended stages, the cell-admission gate, and compliance levels — ships as `protocol-checklist.yaml` for auditing a benchmark against the protocol. A clean-room reproduction from the archived tarball (35/35 raw-data checksums; byte-for-byte regeneration of 45 of 50 tables) is logged in `notes/clean-room-reproduction.md`. This is the generic build; the Elsevier submission build is `ist/ist_main.tex`.

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper, and is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used a large language model (Claude, Anthropic) in order to draft and edit the manuscript text and improve its readability and language. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

AI-assisted research software

Separately from the writing above, a large language model (Claude, Anthropic) also assisted the author in implementing and analyzing the benchmark harness. This is research tooling for which the author takes full responsibility, and all AI-assisted analytical code is independently inspectable and verified: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the byte-level correctness cross-check independently confirms that all layers return identical responses before any timing is trusted.

References

[1] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test.*

Verif. Reliab., vol. 24, no. 3, pp. 219–250, 2014. DOI: 10.1002/stvr.1486.

[2] J. Attala and C. Khemapatpapan, "Comparing the performance of prisma, typeorm, and sequelize on postgresql," *J. Comput. Creat. Technol.*, vol. 3, no. 2, pp. 201–213, 2025. DOI: 10.14456/jcct.2025.16. [Online]. Available: https://so13.tci-thaijo.org/index.php/jcct/article/view/2330.

[3] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–27, 2017. DOI: 10.1145/3133876.

[4] V. R. Basili and H. D. Rombach, "The TAME project: Towards improvement-oriented software environments," *IEEE Trans. Softw. Eng.*, vol. 14, no. 6, pp. 758–773, 1988. DOI: 10.1109/32.6156.

[5] A. Bonvoisin, C. Quinton, and R. Rouvoy, "Understanding the performance-energy tradeoffs of object-relational mapping frameworks," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2024, pp. 626–636. DOI: 10.1109/SANER60148.2024.00069.

[6] K. X. Carmo, F. Ferreira, and E. Figueiredo, "Performance evaluation of back-end frameworks: A comparative study," in *Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI)*, ACM, 2024, pp. 1–9. DOI: 10.1145/3658271.3658314.

[7] I. K. Chaniotis, K.-I. D. Kyriakou, and N. D. Tselikas, "Is Node.js a viable option for building modern web applications? a performance evaluation study," *Computing*, vol. 97, no. 10, pp. 1023–1044, 2015. DOI: 10.1007/s00607-014-0394-9.

[8] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, and P. Flora, "CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, ACM, 2016, pp. 666–677. DOI: 10.1145/2950290.2950303.

[9] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, "Detecting performance anti-patterns for applications developed using object-relational mapping," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, ACM, 2014, pp. 1001–1012. DOI: 10.1145/2568225.2568259.

[10] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, "Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks," *IEEE Trans. Softw. Eng.*, vol. 42, no. 12, pp. 1148–1161, 2016. DOI: 10.1109/TSE.2016.2553039.

[11] A. Cheung, S. Madden, and A. Solar-Lezama, "Sloth: Being lazy is a virtue (when issuing database queries)," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ACM, 2014, pp. 931–942. DOI: 10.1145/2588555.2593672.

[12] A. Cheung, A. Solar-Lezama, and S. Madden, "Optimizing database-backed applications with query synthesis," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2013, pp. 3–14. DOI: 10.1145/2491956.2462180.

[13] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychol. Bull.*, vol. 114, no. 3, pp. 494–509, 1993. DOI: 10.1037/0033-2909.114.3.494.

[14] C. Collberg and T. A. Proebsting, "Repeatability in computer systems research," *Commun. ACM*, vol. 59, no. 3, pp. 62–69, 2016. DOI: 10.1145/2812803.

[15] D. Colley, C. Stanier, and M. Asaduzzaman, "The impact of object-relational mapping frameworks on relational query performance," in *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*, Java, C#, Python, PHP — deliberately excludes JavaScript/Node., IEEE, 2018, pp. 47–52. DOI: 10.1109/iccecome.2018.8659222.

[16] M. Collina *et al.*, *Autocannon: Fast http/1.1 benchmarking tool for node.js*, https://github.com/mcollina/autocannon, Version 7.15.0; accessed 10 July 2026., 2023.

[17] T. D. Cook and D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Boston, MA: Houghton Mifflin, 1979, ISBN: 9780395307908.

[18] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, ACM, 2010, pp. 143–154. DOI: 10.1145/1807128.1807152.

[19] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794.

[20] Drizzle Team, *Drizzle orm benchmarks*, https://orm.drizzle.team/benchmarks, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026)., 2024.

[21] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002, ISBN: 978-0-321-12742-6.

[22] M. Fruth, S. Scherzinger, W. Mauerer, and R. Ramsauer, "Tell-tale tail latencies: Pitfalls and perils in database benchmarking," in *Performance Evaluation and Benchmarking (TPCTC 2021)*, ser. Lecture Notes in Computer Science, vol. 13169, Springer, 2022, pp. 119–134. DOI: 10.1007/978-3-030-94437-7_8.

[23] Gel Data (EdgeDB), *Imdbench: Orm benchmarks*, https://github.com/geldata/imdbench, Accessed 10 July 2026., 2024.

[24] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, ACM, 2007, pp. 57–76. DOI: 10.1145/1297027.1297033.

[25] K. Gupta and M. Mathuria, "Improving performance of web application approaches using connection pooling," in *2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, 2017, pp. 355–358. DOI: 10.1109/ICECA.2017.8212833.

[26] S. Harizopoulos, D. J. Abadi, S. Madden, and M. Stonebraker, "OLTP through the looking glass, and what we found there," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, ACM, 2008, pp. 981–992. DOI: 10.1145/1376616.1376713.

[27] W. Hasselbring, "Benchmarking as empirical standard in software engineering research," in *Proceedings of the Evaluation and Assessment in Software Engineering (EASE)*, ACM, 2021, pp. 457–462. DOI: 10.1145/3463274.3463361.

[28] K. Huppler, "The art of building a good benchmark," in *Performance Evaluation and Benchmarking (TPCTC 2009)*, ser. Lecture Notes in Computer Science, vol. 5895, Springer, 2009, pp. 18–30. DOI: 10.1007/978-3-642-10424-4_3.

[29] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, and M. A. Ibrahim, "Optimizing asynchronous performance in Node.js with Express and PostgreSQL," *Procedia Comput. Sci.*, vol. 269, pp. 172–181, 2025. DOI: 10.1016/j.procs.2025.08.270.

[30] C. Ireland, D. Bowers, M. Newton, and K. Waugh, "A classification of object-relational impedance mismatch," in *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA)*, IEEE, 2009, pp. 36–43. DOI: 10.1109/DBKDA.2009.11.

[31] Z. M. Jiang and A. E. Hassan, "A survey on load testing of large-scale software systems," *IEEE Trans. Softw. Eng.*, vol. 41, no. 11, pp. 1091–1118, 2015. DOI: 10.1109/TSE.2015.2445340.

[32] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, ACM, 2013, pp. 63–74. DOI: 10.1145/2464157.2464160.

[33] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, *et al.*, "Preliminary guidelines for empirical research in software engineering," *IEEE Trans. Softw. Eng.*, vol. 28, no. 8, pp. 721–734, 2002. DOI: 10.1109/TSE.2002.1027796.

[34] S. Kounev, K.-D. Lange, and J. von Kistowski, *Systems Benchmarking: For Scientists and Engineers*. Springer, 2020. DOI: 10.1007/978-3-030-41705-5.

[35] B. Krishnamachari, *How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results*, https://arxiv.org/abs/2605.00428, 2026.

[36] P. Kuffeł and B. Walter, "Changes in Node.js server-side application performance characteristics over time under load," in *Advances in Information Systems Development*, ser. Lecture Notes in Information Systems and Organisation, vol. 77, Springer, 2025, pp. 275–294. DOI: 10.1007/978-3-031-87880-0_14.

[37] C. Laaber, J. Scheuner, and P. Leitner, "Software microbenchmarking in the cloud. how bad is it really?" *Empir. Softw. Eng.*, vol. 24, no. 4, pp. 2469–2508, 2019. DOI: 10.1007/s10664-019-09681-1.

[38] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3348–3361, 2021. DOI: 10.14778/3484224.3484232.

[39] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015. DOI: 10.14778/2850583.2850594.

[40] P. Leitner and C.-P. Bezemer, "An exploratory study of the state of practice of performance testing in Java-based open source projects," in *Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE)*, ACM, 2017, pp. 373–384. DOI: 10.1145/3030207.3030213.

[41] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, ACM, 2014, pp. 1–14. DOI: 10.1145/2670979.2670988.

[42] Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," in *2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*, IEEE, 2013, pp. 15–19. DOI: 10.1109/PACRIM.2013.6625441.

[43] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, and H. Plattner, "Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies," in *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS)*, 2017. DOI: 10.24251/HICSS.2017.592.

[44] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2009, pp. 265–276. DOI: 10.1145/1508244.1508275.

[45] A. V. Papadopoulos, L. Versluis, A. Bauer, *et al.*, "Methodological principles for reproducible performance evaluation in cloud computing," *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1528–1543, 2021. DOI: 10.1109/TSE.2019.2927908.

[46] Prisma, *Query performance benchmarks*, https://benchmarks.prisma.io/, Prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026)., 2024.

[47] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair benchmarking considered difficult: Common pitfalls in database performance testing," in *Proceedings of the Workshop on Testing Database Systems (DBTest)*, ACM, 2018, pp. 1–6. DOI: 10.1145/3209950.3209955.

[48] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2020, pp. 667–682.

[49] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley, 2012, ISBN: 9780321826626.

[50] S. V. Salunke and A. Ouda, "A performance benchmark for the PostgreSQL and MySQL databases," *Future Internet*, vol. 16, no. 10, p. 382, 2024. DOI: 10.3390/fi16100382.

[51] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2006. [Online]. Available: https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html.

[52] S. Shao, Z. Qiu, X. Yu, *et al.*, "Database-access performance antipatterns in database-backed web applications," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2020, pp. 58–69. DOI: 10.1109/ICSME46990.2020.00016.

[53] S. E. Sim, S. Easterbrook, and R. C. Holt, "Using benchmarking to advance research: A challenge to software engineering," in *Proceedings of the 25th International Conference on Software Engineering (ICSE)*, IEEE, 2003, pp. 74–83. DOI: 10.1109/ICSE.2003.1201189.

[54] G. Tene, *How NOT to measure latency*, https://www.infoq.com/presentations/latency-response-time/, Conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026)., 2015.

[55] A. Torres, R. Galante, M. S. Pimenta, and A. J. B. Martins, "Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design," *Inf. Softw. Technol.*, vol. 82, pp. 1–18, 2017. DOI: 10.1016/j.infsof.2016.09.009.

[56] A. Turcotte, M. D. Shah, M. W. Aldrich, and F. Tip, "DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript," in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, ACM, 2022, pp. 774–785. DOI: 10.1145/3510003.3510097.

[57] L. M. Vaquero, L. Rodero-Merino, and R. Buyya, "Dynamically scaling applications in the cloud," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 41, no. 1, pp. 45–52, 2011. DOI: 10.1145/1925861.1925869.

[58] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, 2nd ed. Berlin, Heidelberg: Springer, 2012. DOI: 10.1007/978-3-642-29044-2.

[59] C. Yan, A. Cheung, J. Yang, and S. Lu, "Understanding database performance inefficiencies in real-world web applications," in *Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM)*, ACM, 2017, pp. 1299–1308. DOI: 10.1145/3132847.3132954.

[60] J. Yang, P. Subramaniam, S. Lu, C. Yan, and A. Cheung, "How not to structure your database-backed web applications: A study of performance bugs in the wild," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, ACM, 2018, pp. 800–810. DOI: 10.1145/3180155.3180194.

[61] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, and M. Stonebraker, "Staring into the abyss: An evaluation of concurrency control with one thousand cores," *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 209–220, 2014. DOI: 10.14778/2735508.2735511.

[62] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, and R. R. Siswanto, "Optimizing database access strategy: A performance analysis comparison of raw sql and prisma orm," *Procedia Comput. Sci.*, vol. 269, pp. 1201–1210, 2025. DOI: 10.1016/j.procs.2025.09.061.

[63] S. Zhadko-Bazilevych, "Analysis of ORM framework approaches for Node.js," *J. Comput. Sci. Inst.*, vol. 37, pp. 426–430, 2025. DOI: 10.35784/jcsi.7951.

[64] P. van Zyl, D. G. Kourie, L. Coetzee, and A. Boake, "The influence of optimisations on the performance of an object relational mapping tool," in *Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT)*, ACM, 2009, pp. 150–159. DOI: 10.1145/1632149.1632169.